

Assignment - 3

Name: Divyansh Bhardwaj

Registration no. - RA2211026030052

Branch - CSE AI-ML

Year/Sem/Sec - 4th / 7th / 'A'

Date - 15/10/2025

Subject - Deep Learning

Q1. Differentiate between One-Hot Representation and Distributed Representation of words. Why are distributed representations preferred in modern NLP models?

Answer:

One-Hot Representation and Distributed Representation are two distinct methods for converting words into numerical vectors that can be processed by machine learning models.

One-Hot Representation

A one-hot representation is a sparse, high-dimensional binary vector. In this scheme, each word in the vocabulary is assigned a unique integer index. The vector for a specific word consists of all zeros, except for a single '1' at the index corresponding to that word.

- Characteristics:
 - High Dimensionality: The length of the vector is equal to the size of the entire vocabulary, which can be tens of thousands or more.
 - Sparsity: The vector is mostly filled with zeros.
 - No Semantic Relationship: One-hot vectors are orthogonal to each other (their dot product is always zero). This means the model cannot infer any similarity between words. For example, the vectors for "king" and "queen" are no more similar than the vectors for "king" and "apple."

Example: For a vocabulary of [apple, king, man, queen], the representations would be:

- apple: [1, 0, 0, 0]
- king: [0, 1, 0, 0]
- queen: [0, 0, 0, 1]

Distributed Representation (Word Embeddings)

A distributed representation, also known as a word embedding, is a dense, low-dimensional, continuous-valued vector. In this scheme, a word is represented by a vector where each dimension contributes to defining its meaning. The values in the vector are learned from data.

- Characteristics:
 - Low Dimensionality: The vector length is a hyperparameter, typically ranging from 50 to 300, which is much smaller than the vocabulary size.
 - Density: The vector is composed of real-valued numbers, not zeros.
 - Captures Semantic Relationship: Words with similar meanings will have similar vectors. The similarity can be measured using metrics like cosine similarity. This allows the model to capture relationships like "king is to queen as man is to woman."

Example: In a hypothetical 3-dimensional space:

- king: [0.98, 0.21, 0.15]
- queen: [0.95, -0.85, 0.18] (similar to 'king' but different along the gender axis)
- apple: [-0.54, 0.05, 0.77] (very different from 'king' and 'queen')

Why Distributed Representations are Preferred:

Distributed representations are overwhelmingly preferred in modern Natural Language Processing (NLP) models for several critical reasons:

1. **Semantic Understanding:** They capture the meaning and context of words. This allows models to generalize better. If a model learns something about the word "cat," it can apply that knowledge to the word "kitten" because their vectors are close in the embedding space.
2. **Dimensionality Reduction:** They solve the "curse of dimensionality" associated with one-hot vectors. A smaller, denser vector is far more computationally efficient and requires significantly fewer parameters for the model to learn.
3. **Generalization Power:** By understanding word similarities, models can make reasonable predictions even for words they haven't seen in a specific context during training, as long as they have seen semantically similar words.
4. **Contextualization:** Modern models like Transformers can create contextualized distributed representations, where the vector for a word changes depending on the sentence it's in (e.g., the vector for "bank" is different in "river bank" vs. "money bank"). This is impossible with one-hot encoding.

Q2. Explain the Continuous Bag of Words (CBOW) and Skip-Gram models in Word2Vec. How do they capture semantic relationships between words?

Answer:

Word2Vec is a foundational technique for learning distributed representations of words (word embeddings). It uses a simple neural network with a single hidden layer to perform a proxy prediction task. The learned weights of this hidden layer become the word vectors. The two main architectures within Word2Vec are Continuous Bag of Words (CBOW) and Skip-Gram.

Continuous Bag of Words (CBOW)

- **Architecture:** The CBOW model predicts a target word based on its surrounding context words.
- **Working:** The model takes a "bag of words" from the context (e.g., the two words before and two words after the target) as input. The one-hot vectors of these context words are retrieved, their corresponding embeddings are looked up, and these

embedding vectors are averaged. This averaged vector is then fed through a neural network to predict the target word.

- Analogy: It's like a fill-in-the-blanks task: The quick brown [] jumps over the lazy dog. The model uses (The, quick, brown, jumps) to predict fox.
- Characteristics: CBOW is generally faster to train and tends to perform better for frequent words.

Skip-Gram

- Architecture: The Skip-Gram model does the opposite of CBOW. It predicts the surrounding context words given a single target word.
- Working: The model takes a single input word. It looks up its embedding vector and feeds it through a neural network to predict multiple output words—the words that are likely to appear in its context.
- Analogy: It's like guessing the surrounding words: Given the word 'fox', predict that 'quick', 'brown', 'jumps', and 'over' are nearby.
- Characteristics: Skip-Gram is slower to train but is considered to be more effective, especially for capturing the meaning of rare words and for building more accurate embeddings overall.

How They Capture Semantic Relationships

Both CBOW and Skip-Gram capture semantic relationships based on the distributional hypothesis, which states that "a word is characterized by the company it keeps." Words that frequently appear in similar contexts will have similar meanings.

The process works as follows:

1. The Proxy Task: The models are not explicitly taught semantics. They are simply trained to get better at their respective prediction tasks (predicting the target word or the context words).
2. Weight Updates: During training, if two different words (e.g., "cat" and "kitten") often appear with the same context words (like "cute," "pet," "purring"), the model will adjust their embedding vectors to be closer together to minimize the prediction error.
3. Emergence of Structure: As the model processes a massive corpus of text, this continuous adjustment of weights causes the embedding vectors to organize themselves in a high-dimensional space that reflects their semantic and syntactic relationships.
4. Vector Arithmetic: This learned structure is so powerful that it enables vector arithmetic. The most famous example is $\text{vector('king')} - \text{vector('man')} + \text{vector('woman')}$, which results in a vector very close to vector('queen') . This demonstrates that the model has learned concepts like royalty and gender as directions in the embedding space.

Q3. Discuss the architecture and working of Convolutional Neural Networks (CNNs). Explain the importance of kernel filters, padding, stride, and max pooling in feature extraction.

Answer:

Architecture and Working of CNNs

A Convolutional Neural Network (CNN) is a specialized type of deep neural network designed primarily for processing grid-like data, such as images. Its architecture is inspired by the human visual cortex and is highly effective at automatically learning a hierarchy of features from the input.

A typical CNN architecture consists of several key layers stacked together:

1. **Input Layer:** Holds the raw pixel values of the image (e.g., 32x32x3 for a color image).
2. **Convolutional (Conv) Layer:** The core building block of a CNN. It performs the convolution operation to extract features.
3. **Activation Layer:** Applies a non-linear activation function (usually ReLU) to the output of the convolutional layer.
4. **Pooling Layer:** Downsamples the feature maps, reducing their dimensionality and computational complexity.
5. **Fully Connected (FC) Layer:** After several Conv and Pooling layers, the feature maps are flattened into a 1D vector and fed into one or more FC layers for classification.
6. **Output Layer:** Produces the final output (e.g., class probabilities using a Softmax function).

Working: The network learns a hierarchy of features. The first few Conv layers learn simple features like edges, corners, and colors. Subsequent layers combine these simple features to learn more complex patterns like textures, shapes, and eventually, entire objects.

Importance of Key Components in Feature Extraction

1. **Kernel Filters (or Kernels):**
 - What they are: Small, learnable matrices of weights (e.g., 3x3 or 5x5) that slide, or "convolve," across the input image or feature map.
 - Importance: They are the feature detectors of the CNN. Each kernel is responsible for learning to detect a specific pattern. For instance, one kernel might learn to activate when it sees a vertical edge, another a horizontal edge, and another a specific color. The values of these kernels (the weights) are what the network learns during training.
2. **Padding:**
 - What it is: The process of adding extra pixels (usually with a value of zero) around the border of the input image before the convolution operation.
 - Importance:

- Preserves Spatial Dimensions: Without padding, each convolution operation would shrink the size of the feature map. Padding allows the output feature map to have the same spatial dimensions as the input, which is crucial for building deep networks.
- Improves Edge Feature Extraction: It allows the kernel to process the pixels at the very edge of the image more effectively, preventing information loss at the borders.

3. Stride:

- What it is: The number of pixels the kernel filter shifts at a time as it moves across the input image. A stride of 1 moves the filter one pixel at a time; a stride of 2 moves it two pixels at a time.
- Importance: Stride controls the size of the output feature map. Using a stride greater than 1 has a downsampling effect, reducing the spatial dimensions and the amount of computation required.

4. Max Pooling:

- What it is: A downsampling technique. It slides a window (e.g., 2x2) over the feature map and, from the pixels within that window, outputs only the maximum value.
- Importance:
 - Reduces Dimensionality: It drastically reduces the number of parameters and computations in the network, controlling overfitting.
 - Provides Translation Invariance: By taking the maximum activation, the pooling layer makes the feature representation more robust to small shifts or translations of the feature in the input image. It focuses on *whether* a feature is present, not *exactly where* it is.

Q4. Compare the architectures and key contributions of AlexNet, GoogLeNet (Inception), and ResNet. How did these models influence the development of deep convolutional networks?

Answer:

AlexNet, GoogLeNet, and ResNet are three landmark CNN architectures that won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) and fundamentally shaped the evolution of deep learning for computer vision.

Comparison of Architectures and Contributions

Feature	AlexNet (2012)	GoogLeNet / Inception (2014)	ResNet (2015)
Depth	8 layers (considered deep at the time)	22 layers	34, 50, 101, 152+ layers

Key Contribution	Proved that deep CNNs could achieve state-of-the-art results on a challenging dataset.	Introduced the Inception Module, allowing for efficient deep networks by using multi-scale processing.	Introduced Residual Connections (Skip Connections), making it possible to train extremely deep networks (100+ layers) effectively.
Architecture Innovation	* Used ReLU activation function (faster training). * Implemented Dropout to combat overfitting. * Used extensive data augmentation.	* Inception Module: Performed 1x1, 3x3, and 5x5 convolutions in parallel and concatenated their outputs. * Used 1x1 convolutions for dimensionality reduction, making the network computationally cheaper.	* Residual Block: Added the input of a block (x) to its output (F(x)), so the network learns a residual function $F(x) = H(x) - x$. This creates a "shortcut" for the gradient flow.
Problem Solved	Showed that scaling up CNNs with more data and compute was a viable path to success.	Addressed the problem of choosing the right kernel size and creating deeper networks without a massive increase in computational cost.	Solved the degradation problem, where adding more layers to a plain deep network caused its performance to worsen.

Influence on the Development of Deep Convolutional Networks

These three models created a clear evolutionary path for CNNs, with each one building on the lessons of its predecessor.

1. AlexNet's Influence:

- The "ImageNet Moment": AlexNet's dominant victory in 2012 was a watershed moment that convinced the computer vision community of the power of deep learning. It triggered a massive shift in research and industry focus towards CNNs.
- Established a Blueprint: It popularized the use of ReLU, dropout, and data augmentation, which became standard practices in nearly all subsequent deep learning models.

2. GoogLeNet's Influence:

- Shift Towards Efficiency: It demonstrated that networks could be made deeper and more powerful without simply increasing the raw number of parameters. The Inception module's design principle of "going wider, not just deeper" and using 1x1 convolutions for efficiency became a key architectural pattern.

- Network-in-Network Idea: It promoted the idea of complex micro-architectures (the Inception module) being repeated to build a larger network.
3. ResNet's Influence:
- Broke the Depth Barrier: ResNet's introduction of residual connections was a revolutionary breakthrough. It elegantly solved the vanishing gradient and degradation problems, making it feasible to train networks of unprecedented depth (hundreds or even thousands of layers).
 - The New Standard: The residual block became the new default building block for deep architectures, not just in computer vision but also in other domains. It showed that the key to depth was not just stacking layers but providing an unimpeded path for information and gradients to flow.

In essence, AlexNet proved that depth works, GoogLeNet showed how to make depth efficient, and ResNet showed how to make networks extremely deep. Together, they form the backbone of modern deep convolutional networks.

Q5. What are Recurrent Neural Networks (RNNs)? Explain the concept of Backpropagation Through Time (BPTT) and how it leads to exploding and vanishing gradient problems.

Answer:

Recurrent Neural Networks (RNNs)

A Recurrent Neural Network (RNN) is a type of neural network specifically designed to process sequential data, where the order of elements is important. This includes data like text, speech, and time-series financial data.

The defining feature of an RNN is its recurrent loop. At each time step t , the network takes an input x_t and its own hidden state from the previous time step, h_{t-1} , to produce an output y_t and update its hidden state to h_t . This hidden state h_t acts as the network's "memory," carrying information from past elements of the sequence to influence future predictions.

Backpropagation Through Time (BPTT)

Backpropagation Through Time (BPTT) is the algorithm used to train RNNs. Since a standard backpropagation algorithm cannot handle the recurrent (cyclic) nature of an RNN, BPTT works by first unrolling the network through time.

1. Unrolling the Network: The RNN is conceptually unrolled into a deep feedforward network, with one layer for each time step in the sequence. The weights are shared across all these time steps.
2. Standard Backpropagation: The standard backpropagation algorithm is then applied to this unrolled network.

3. Accumulating Gradients: The error from each time step is propagated backward through the unrolled network. The gradients for the shared weights are summed up across all time steps.
4. Weight Update: Finally, the weights are updated using the accumulated gradients.

Exploding and Vanishing Gradient Problems

The core issue with BPTT arises from the repeated multiplication of gradients through the recurrent connections during the backward pass. The gradient at an early time step is a product of the gradients from all later time steps. This involves repeatedly multiplying by the recurrent weight matrix W_{hh} .

1. Vanishing Gradients:
 - Cause: If the values in the recurrent weight matrix are small (specifically, if its largest eigenvalue is less than 1), the repeated multiplication during the backward pass will cause the gradient to shrink exponentially.
 - Effect: By the time the gradient reaches the early time steps, it becomes virtually zero. This means the network is unable to learn long-range dependencies. It cannot update its weights based on errors that occurred many time steps in the past. This is the more severe and common problem in RNNs.
2. Exploding Gradients:
 - Cause: If the values in the recurrent weight matrix are large (its largest eigenvalue is greater than 1), the repeated multiplication will cause the gradient to grow exponentially.
 - Effect: The gradient becomes massive, leading to huge, unstable weight updates. The training process becomes unstable, and the loss often becomes NaN (Not a Number). This problem is easier to detect and can be mitigated by a technique called gradient clipping, where the gradient is capped at a certain threshold.

These gradient problems made it very difficult for simple RNNs to be effective on tasks requiring long-term memory, which led to the development of more sophisticated architectures like LSTMs and GRUs that use gating mechanisms to better control the flow of gradients.

Q6. Describe the Encoder–Decoder model and its role in Sequence-to-Sequence learning tasks such as machine translation. How does Attention improve the performance of such models?

Answer:

The Encoder-Decoder Model

The Encoder-Decoder model is a powerful neural network architecture designed for Sequence-to-Sequence (Seq2Seq) tasks. These are tasks where both the input and the output

are sequences of variable length, such as machine translation (translating a sentence from one language to another), text summarization, and chatbot responses.

The architecture consists of two main components, typically implemented with Recurrent Neural Networks (like LSTMs or GRUs):

1. The Encoder:

- Its role is to process the entire input sequence, one element at a time.
- It learns to compress the information from the entire input sequence into a single, fixed-size vector known as the context vector (or "thought vector"). This context vector is usually the final hidden state of the encoder RNN.
- The goal of the encoder is to create a meaningful numerical summary of the input sequence.

2. The Decoder:

- Its role is to generate the output sequence, one element at a time.
- It is initialized with the context vector from the encoder. This vector provides the decoder with the summary of what it needs to generate.
- At each time step, the decoder takes the previously generated output element as input and generates the next element of the output sequence, conditioned on the context vector and its own hidden state.

The Bottleneck Problem

The primary limitation of this basic Encoder-Decoder model is the fixed-size context vector. It becomes a bottleneck, as it must encapsulate the meaning of the entire input sequence, regardless of its length. For long and complex sentences, it is very difficult to compress all the necessary information into this single vector, leading to information loss and poor performance.

How Attention Improves Performance

The Attention Mechanism was introduced to solve this bottleneck problem and dramatically improve the performance of Seq2Seq models.

Instead of relying on a single context vector, attention allows the decoder to "look back" and focus on different parts of the original input sequence at each step of its output generation.

Here's how it works:

1. Access to All Encoder States: The decoder is given access to *all* the hidden states produced by the encoder for every input word, not just the final one.
2. Calculate Attention Scores: At each step of generating an output word, the decoder compares its current hidden state with all of the encoder's hidden states to compute a set of attention scores. These scores determine how relevant each input word is to the current output word being generated.

3. Compute Attention Weights: The attention scores are passed through a softmax function to create a probability distribution called attention weights. These weights sum to 1.
4. Create a Dynamic Context Vector: A weighted sum of all the encoder hidden states is calculated using these attention weights. This creates a new, dynamic context vector that is tailored specifically for generating the current output word.

The Impact:

Attention frees the model from the constraint of the single fixed-size context vector. It allows the decoder to selectively focus on the most relevant parts of the input source. For example, when translating a sentence, as the decoder generates a French word, the attention mechanism can focus heavily on the corresponding English word(s), significantly improving translation accuracy, especially for long sentences.

Q7. What is the Attention Mechanism? Explain different types of attention (e.g., spatial attention, hierarchical attention) and discuss their applications in image captioning and language modeling.

Answer:

The Attention Mechanism

The Attention Mechanism is a powerful concept in deep learning that mimics human cognitive attention. It allows a neural network to dynamically focus on the most relevant parts of an input when producing a specific output. Instead of treating the entire input as a single unit, attention enables the model to assign different "weights" or levels of importance to different parts of the input, and then compute a representation based on a weighted combination of these parts. This makes the model more efficient, interpretable, and effective, especially for long and complex inputs.

Different Types of Attention

1. Spatial Attention (in Computer Vision):
 - Concept: This type of attention focuses on *where* to look in an image. Instead of giving equal importance to all pixels, spatial attention learns to assign higher weights to the most salient or relevant regions of the image for a given task.
 - Application - Image Captioning: In an image captioning model, the task is to generate a sentence describing an image. When the model is generating a specific word in the caption, spatial attention allows it to focus on the corresponding region in the image. For example:
 - When generating the word "dog," the attention mechanism would assign high weights to the pixels that make up the dog.
 - When generating the word "frisbee," it would shift its focus to the pixels representing the frisbee.

This allows the model to generate much more accurate and detailed captions.

2. Hierarchical Attention (in NLP):

- Concept: This type of attention operates at multiple levels of granularity, reflecting the hierarchical structure of data like text. For a document, the model first attends to words to build sentence representations, and then attends to sentences to build a document representation.
- Application - Language Modeling / Document Classification: In a language model that needs to understand a long document, hierarchical attention can be very effective.
 - Word-level Attention: First, for each sentence, the model identifies the most important words to form a meaningful sentence vector.
 - Sentence-level Attention: Next, it takes all the sentence vectors and applies attention again to identify the most important sentences that contribute to the overall meaning or classification of the document. This allows the model to efficiently process long texts while focusing on the most informative parts at both the word and sentence levels.

Other types of attention include self-attention, which is the cornerstone of the Transformer architecture, where elements within a single sequence attend to each other to build a context-aware representation.

Q8. Define Generative Adversarial Networks (GANs). Explain their architecture, training process, and mention at least two real-world applications of GANs.

Answer:

Definition of Generative Adversarial Networks (GANs)

A Generative Adversarial Network (GAN) is a class of unsupervised machine learning models designed to generate new data that is convincingly similar to a given training dataset. It was introduced by Ian Goodfellow and his colleagues in 2014 and has been a major breakthrough in the field of generative modeling.

Architecture

A GAN's architecture is based on a two-player, zero-sum game between two competing neural networks:

1. The Generator (G):

- Role: The "counterfeiter" or "artist."
- Process: It takes a random noise vector as input and tries to transform it into a sample of data (e.g., an image) that looks like it could have come from the real training dataset.
- Goal: To generate data that is so realistic that it can fool the Discriminator.

- 2.
3. The Discriminator (D):
 - Role: The "police" or "art critic."
 - Process: It is a standard binary classifier. It takes a sample of data (either a real one from the training set or a fake one from the Generator) as input and tries to determine whether it is real or fake.
 - Goal: To correctly distinguish between real and fake data.
- 4.

Training Process

The Generator and Discriminator are trained simultaneously in an adversarial process:

1. Train the Discriminator: A batch of real data and a batch of fake data (generated by the current Generator) are shown to the Discriminator. The Discriminator's weights are updated via backpropagation to improve its ability to tell them apart (e.g., to output 1 for real and 0 for fake).
2. Train the Generator: A new batch of fake data is created by the Generator. This fake data is passed to the Discriminator. The error is calculated based on how well the Generator fooled the Discriminator (i.e., how close the Discriminator's output was to 1). The gradients are then backpropagated *only through the Generator's weights*, updating them to make it better at fooling the Discriminator in the next round.
3. Repeat: These two steps are repeated iteratively. Over time, the Generator gets better at creating realistic data, and the Discriminator gets better at detecting fakes. This competition drives both networks to improve until they reach an equilibrium where the Generator produces samples that are indistinguishable from the real data.

Two Real-World Applications of GANs

1. Image Generation and Art Creation: GANs, particularly advanced versions like StyleGAN, can generate incredibly realistic, high-resolution images of things that do not exist, such as human faces, animals, or landscapes. This technology is used in art creation, video game development for generating textures and characters, and in the fashion industry for designing new products.
2. Image-to-Image Translation and Data Augmentation: GANs can be used to translate an image from one domain to another. For example, they can turn a satellite image into a map (pix2pix), a sketch into a photorealistic image, or a summer scene into a winter one (CycleGAN). In medical imaging, this can be used to generate synthetic MRI scans from CT scans or to augment limited datasets of rare diseases to improve the training of diagnostic models.